Improving Java Application Performance and Scalability by Reducing Garbage Collection Times and Sizing Memory

by Nagendra Nagarajayya and J. Steven Mayer
July 2002

Abstract

As Java technology becomes more and more pervasive in the telecommunications (telco) industry, understanding the behavior of the garbage collector becomes more important. Typically, telco applications are near-real-time applications. Delays measured in milliseconds are not usually a problem, but delays of hundreds of milliseconds, let alone seconds, can spell trouble for applications of this kind. Quite simply, sub-optimal performance translates directly into loss of revenue.

Critics usually point to inefficient garbage collection (GC) as a reason why these types of applications cannot perform well if developed using Java technology. This paper challenges that accusation, and asserts that developers can improve performance significantly, and describes analytical and modeling strategies that will result in acceptable application performance.

Developers must first get a solid background in garbage collection, examine the different algorithms and implementations, come to understand how they affect the application, and learn about the GC problems Java applications have historically faced. Then they can construct a mathematical model to predict and tune application behavior.

By using the output from the –verbose:gc option in the Java Virtual Machine (JVM) and the GC analyzer, developers can analyze object creation, object lifetimes, and garbage collection of these objects. Then they can use the knowledge they've gained to improve the performance and scalability of their applications, using advanced GC algorithms to reduce application pause times to acceptable levels.

The information obtained from the mathematical model enables developers to find the optimum operating conditions for an application, to discover how to determine the optimal size for young and old-generation heaps, and to use an advanced concurrent collector to all but eliminate old-generation collection pause times from an application.

Table of Contents

1. Garbage Collection in Java Applications

The Java programming language is inherently object-oriented [3], and includes automatic garbage collection. Garbage collection is the process of reclaiming memory taken up by unreferenced objects. Unreferenced objects are ones the application can no longer reach because all references to them have gone out of extent.

Traditional languages like C and C++ do not have automatic garbage collection. Developers must both allocate and free memory manually, by using the malloc() and free() functions in C for instance, or the new and delete operators in C++. In Java applications, heap memory is allocated by the new operator, but developers need not free this memory explicitly. Instead, the runtime system routinely determines which objects still have valid references (live objects) and which are unreferenced (dead objects), and automatically frees up the memory allocated to the dead objects. This process is aptly called garbage collection.

Copy

```
public class GarbageCollectionExample
{
public void showGarbageCollection()
{
String gcObject = new String("Hello, World\n");                 //  Step 3
return;                                                         //  Step 4
}
public static void main(String args[])
{
GarbageCollectionExample gce = new GarbageCollectionExample();  //  Step 1
gce.showGarbageCollection();                                    //  Step 2
//...
}
}
```

This code snippet shows how an object becomes unreferenced in a Java program.

- In Step 1, an object is created, and associated with the name `gce`.
- In Step 2, gce's method `showGarbageCollection()` is called.
- In Step 3, another object is created, and associated with the name `gcObject`.
- In Step 4, the reference gcObject ceases to exist when the return finishes execution of the `showGarbageCollection()` method, and control returns to `main()`.

At this point, we no longer have a reference to the `String` we knew as `gcObject`. It is now an unreferenced, "dead" object, and can be collected. The object `gce` is still referenced in the `main` method, and is not collectable.

2. The Garbage Collector

Note that the discussions of garbage collectors, the various GC algorithms, and modeling in the following sections focus on Sun's implementations of the Java Virtual Machine.

The Java runtime system's garbage collector runs as a separate thread. As the application allocates more and more objects, it depletes the amount of heap memory available. When this drops below a threshold percentage, the garbage collection process begins. The garbage collector stops all other runtime threads. It marks each object as live or dead, and reclaims the space taken up by the dead objects.

There are many different types of garbage collectors. Each is based on a different algorithm and exhibits different behavior. They range from simple reference-counting collectors to very advanced generational collectors. Both the algorithm chosen and the implementation can affect the behavior of the garbage collector. Here are some terms that refer to the implementation of a particular collector – note these are not mutually exclusive:

- Stop-the-world – Stops all application threads while it works.
- Concurrent – Allows application threads to run while it works.
- Parallel – Multiple threads work on collecting at the same time.

A few of the traditional collectors are discussed below.

2.1 Copying Collector

A copying collector employs two or more storage areas to create and destroy objects. If it uses only two storage areas, they are called "semi-spaces." One semi-space (the "from-space") is used to create objects, and once it is full, the live objects are copied to the second semi-space (the "to-space"). Memory is compacted at no cost because only live objects are copied, and they are stored linearly. The semi-spaces now switch roles: new objects are created in the second one until it is full, at which point live objects are copied to the first. The semi-spaces exchange the from-space and to-space roles again and again. Dead objects are not freed explicitly; they're simply overwritten by new objects.

In a JVM, a copying collector is a stop-the-world collector. Nevertheless, it is extremely efficient because it traverses the object list and copies the objects in a single cycle, and thus simultaneously collects the garbage and compacts the heap. The time to collect the semi-space, the "pause duration," is directly proportional to the total size of live objects.

2.2 Mark and Compact Collector

In a mark-and-compact (more briefly, "mark-compact") collector, objects are created in a contiguous space, the heap. Once the free space falls below a threshold, the collector begins a marking phase, in which it traverses all objects from the "roots," marking each as either live or dead. After the marking phase ends, the compacting phase begins, in which the collector compacts the heap by copying live objects to a new contiguous area. In a JVM, a mark-compact collector is a stop-the-world collector. It suspends all of the application threads until collection is complete and the memory is reorganized, and then restarts them.

2.3 Mark and Sweep Collector

The marking phase of a mark-and-sweep ("mark-sweep") collector is the same as that of a mark-compact collector. When the marking phase is complete, the space each dead object occupies is added to a free list. Contiguous space occupied by dead objects is combined to make larger segments of free memory, but because a mark-sweep collector does not compact the heap, memory has a tendency to fragment over time.

2.4 Incremental Collector

An incremental collector divides the heap into small fixed-size blocks and allocates the data among them. It runs only for brief periods of time, leaving more processor time available for the application's use. It collects the garbage in only one block at a time, using the train algorithm [7]. The train algorithm organizes the blocks into train-cars and trains. In each collection cycle, the collector checks the cars and trains. If the train to which a car belongs contains only garbage, then the GC collects the entire train. If a car has references from other cars, then the GC copies objects to the respective cars, and, if the destination cars are full, it creates new cars as needed. Because the incremental collector pauses the application threads for only brief periods of time, the net effect is near-pauseless application execution.

2.5 Generational Garbage Collection

In most object-oriented languages, including the Java programming language, most objects have very short lifetimes, while a small percentage of them live much longer . Of all newly allocated heap objects, 80-98 percent die within a few million instructions . A large percentage of the surviving objects continue to survive for many collection cycles, however, and must be analyzed and copied at each cycle. Hence, the garbage collector spends most of its time analyzing and copying the same old objects repeatedly, needlessly, and expensively.

To avoid this repeated copying, a generational GC divides the heap into multiple areas, called generations, and segregates objects into these areas by age. In younger generations objects die more quickly and the GC collects them more frequently, while in older generations objects are collected less often. Once an object survives a few collection cycles, the GC moves it to an older generation, where it will be analyzed and copied less often. This generational copying reduces GC costs.

The GC may employ different collection algorithms in different generations. In younger generations, objects are ephemeral, and both space requirements and numbers of objects needing copying tend to be small, so a copying collector is extremely efficient. In older generations, objects tend to be more numerous and longer-lived, and copying costs make copying collectors (2.2) prohibitively expensive, hence mark-compact (2.1) or mark-sweep (2.3) collectors are preferred.

2.5.1 Generational Garbage Collection in Java Applications

Generational collection was introduced to the JVM in v1.2. The heap was divided into two generations, a young generation that used two semi-spaces and a copying collector, and an old generation that used a mark-compact or mark-sweep collector. It also offered an advanced collector, the concurrent, incremental mark-and-sweep ("concurrent inc-mark-sweep") collector, (see 2.6 Advanced Collectors in Java).

In the 1.3 and later JVMs [4], the heap is again divided into generations – by default, two generations. The young generation uses a copying collector, while the old generation uses a mark-compact collector. The 1.3 JVM also offers an incremental collector, introducing an optional intermediate generation between the young and old generations. Advanced collectors, like the concurrent inc-mark-sweep collector, are not available in 1.3 but are available in 1.2.2_07, and from the 1.4.1 JVM onwards.

More information about the collectors available from Sun can be found at: Hot Spot.

2.6 Advanced Collectors in Java Applications

The Java platform provides an advanced collector, the concurrent inc-mark-sweep collector. A concurrent collector takes advantage of its dedicated thread to enable both garbage collection and object allocation/modification to happen at the same time. It uses external bitmaps [1] to manage older-generation memory, and "card tables" [2] to interact with the younger-generation collector. The bitmaps are used to scan the heap and mark live objects concurrently, in a "marking phase." Once the marking is complete, the unmarked objects are deallocated in a concurrent "sweeping phase." The collector does most of its work concurrently, suspending application execution only briefly.

Note: If "the rate of creation" of objects is too high, and the concurrent collector is not able to keep up with the concurrent collection, it falls back to the traditional mark-sweep collector.

2.7 The Sun Exact VM

The research for this paper was done using the 1.2.2_08 version of the JDK (Java 2 SDK, Standard Edition), on Solaris, with the advanced concurrent inc-mark-sweep collector.

The Exact VM is a high-performance Java virtual machine developed by Sun Microsystems. It features high-performance, exact memory management. The memory system is separated from the rest of the VM by a well-defined GC interface. This interface allows various garbage collectors to be "plugged in". It also features a framework to employ generational collectors.

Collectors and Usage in JDK 1.2.2_08

JDK 1.2.2_08 supports generational collection. Two generations are available, young and old. The young generation employs a copying collector, while the old generation uses the mark-compact collector. The old-generation collector may be replaced with others.

2.7.1.1 JDK 1.2.2_08 Default Usage

```
java program
```

By default the young generation uses 2 MB for each semi-space, for a total of 4 MB. The old generation defaults to 2 MB of initial heap and 24 MB of maximum heap. These defaults may be overridden as described below.

2.7.1.2 Using the Xms and Xms Switches

The old generation's default heap size can be overridden by using the `-Xms` and `-Xmx` switches to specify the initial and maximum sizes respectively:

```
java -Xms
<initial size> -Xmx <maximum size> program
```

For example:

```
java
 -Xms64m -Xmx128m program
```

2.7.1.3 Using the genconfig Switch

The `genconfig` switch can be used to specify the heap sizes for the young and old generations explicitly. It also allows an old-generation collector to be specified. (The `genconfig` option makes the `-Xms` and `-Xmx` options unnecessary, but if used with `genconfig` option `-Xmx` is still honored as a means to limit the old-generation heap size.) The syntax for `genconfig` is as follows:

```
java
 -Xgenconfig   :<initial young size>,   <max young size>,
 semispaces[,promoteall]   :<initial old size>,   <max old size>   [,<collector>]
program
```

For example:

```
java
 -Xgenconfig:8m,8m,semispaces:128m,128m
```

```
program
```

In this example `:8m,8m,semispaces` sets the initial and maximum sizes of the young generation's semi-spaces to 8 MB each, and `:128m,128m` specifies the initial and maximum sizes of the old heap to be 128 MB.

The young generation always uses a copying collector, but the default mark-compact collector for the old generation can be overridden, for example with an incremental mark-sweep collector, thus:

```
java -Xgenconfig:8m,8m,semispaces:128m,128m,
 incmarksweep program
```

By default, the young-generation copying collector copies live objects from one semi-space to the other (in a process called "tenuring" [1]) before copying them into the old heap (called "promoting"). The `promoteall` option overrides this default behavior: the young collector will immediately promote all live objects into the old heap during each collection. This variation can be useful for applications in which objects that live through a single young GC are likely to live through many collections. An example of its use:

[1] *Tenuring* is a well-known term, meaning "aging." It usually refers to objects being promoted to the older generation, but in our case it has been used to describe aging taking place in the younger generation. This was done to differentiate the two types of garbage collectors in the younger generation: a copy GC, which copies objects to the "to" semispace, and a promotion GC which copies objects to the older generation.

```
java -Xgenconfig
:8m,8m,semispace,
 promoteall
:128m,128m,incmarksweep
program
```

3. Problems with Java Garbage Collection

Garbage collection makes memory management easier for application developers by eliminating the need to deallocate memory explicitly. This in turn eliminates the possibility of whole classes of errors, including memory leaks and attempts to release or use memory that has not been allocated. Freeing the developer from explicit heap handling increases productivity while making applications robust, but garbage collection comes at a cost. It makes application behavior non-deterministic by introducing latency . It also affects throughput, as collection cycles slow end-to-end performance.

To refine an earlier description: A generational garbage collector runs as a separate thread in the Java runtime system. The collector gets activated when the free threshold in either the young generation or the old generation drops below a certain percentage. The young generation's copying collector stops all application threads while it performs a collection. This stop-the-world behavior introduces pauses into an application run, and the duration of the pause is directly proportional to the size of the heap, and the number of live objects.

The less sophisticated collectors for old generations, such as mark-compact and mark-sweep, are also stop-the-world collectors. The duration of the pause is proportional to the "occupied" size of the old heap.

Because the old heap is typically much larger than the young heap, its pauses are longer but less frequent. The duration and frequency of these pauses is difficult to predict because they are affected by many factors, among them:

- Size of the old generation
- Lengths of objects' lifetimes
- Allocation rate
- Which collector is being used

The pauses garbage collection imposes introduce serial behavior into applications that might otherwise be neatly concurrent. The stop-the-world garbage collector in the JVM uses only a single CPU while performing GC, and it inevitably reduces an application's scalability in a multi-processor environment. Using one of the advanced collectors can mitigate this problem.

An incremental collector, which introduces many, very short pauses, can reduce pause duration. The pauses' brevity makes the application seem to run pauselessly, but they may actually degrade overall application performance because each pause still stops-the-world, and, even though it decreases the duration, increasing the frequency usually results in a higher total cost.

The advanced, mostly concurrent, inc-mark-sweep collector greatly reduces the serialization problem, by allowing the application threads to run concurrently with a collection cycle. The pauses introduced by the concurrent collector are very small because the majority of the work is done concurrently. While the concurrent GC is running, though, it makes use of CPU cycles and thus reduces application throughput somewhat — especially that of compute-bound applications.

Garbage collection in young generations remains one of the main obstacles to scalability. No matter which collector is chosen for the old generation, the young generation still uses a single-threaded, stop-the-world copying collector.
A multi-threaded collector for the young generation is expected to mitigate this problem in the near future.

4. How GC Pauses Affect Scalability and Execution Efficiency

Linear scalability of an application running on an SMP(Symmetrical Multi Processing)-based machine is directly limited by the serial parts of the application. The serial parts of a Java application are:

- Stop-the-world GC
- Access to critical sections
- Concurrency overhead such as scheduling, context switching,...
- System and communication overhead

The percentage of time spent in the serial portions determines the maximum scalability that can be achieved on a multi-processor machine and in turn the execution efficiency of an application. Because the young– and old-generation collectors all have at least some single-threaded stop-the-world behavior, GC will be a limiting factor to scalability even when the rest of the application can run completely in parallel. Using the concurrent inc-mark-sweep collector will help reduce this effect, but will not completely remove it.

The scalability and execution efficiency of an application can be calculated using Amdahl's law.

4.1 Amdahl's Law & Efficiency calculations

If S is the percentage of time spent (by one processor) on serial parts of the program and P is the percentage of time spent (by one processor) on parts of the program that could be done in parallel , then:

```
Speedup = (S + P) / (S + P / N)          -> (1)
where N is the number of processors.
This can be reduced to:
Speedup = 1 / (F + (1 - F) / N) -> (2)
where F is the percentage of time spent in the
serial parts of the application
and N is the number of processors.
Speedup = 1 / F                          -> (3)
when N is very large.
So if
 F = 0 (i.e., no serial part),
then speedup = N (the ideal value).
If
 F = 1< (i.e., completely serial),
then speedup = 1
no matter how many processors are used.
The effect of the serial fraction
F on the speedup produced for N = 10:
```

4.1.1 Scaled Speedup

Assuming that the problem size is fixed, then Amdahl's Law can predict the speedup on a parallel machine:

```
Speedup = (S + P) / (S + P / N)                 -> (4)
Speedup = 1 / (F + (1 - F) / N)                 -> (5)
```

More details on Amdahl's Law and scaled speedup can be obtained from [11] .

4.1.2 Efficiency

Execution efficiency is defined as:

```
E = Speedup / N                                 -> (6)
```

Execution efficiency translates directly to the CPU percentage used by an application. For a linear speedup this is 1, or 100%. The higher this number is the better, because it translates to higher application efficiency.

5. A SIP Server, the Benchmark Application

This section provides background on the application that was used to do the research for this paper. It also supplies background information about why this research was performed.

5.1 Problem Overview

Session Initiation protocol (SIP) [18] is a protocol defined by the Internet Engineering Task Force (IETF), used to set up a session between two users in real time. It has many applications, but the one focused on here is setting up a call between users. After a call setup is completed, the SIP portion is complete. The users are then free to pass real-time media (such as voice) between the two endpoints. Note that this portion, the call, does not involve SIP or the servers that are routing SIP call setups.

If this protocol is still unfamiliar, it might help to think of it as akin to Hyper Text Transfer Protocol (HTTP)) [21]. It is a similar, text-based protocol founded on the request/response paradigm. One of the key differences is that SIP supports unreliable transport protocols, such as UDP. When using UDP, SIP applications are responsible for ensuring that packets reach their destination. This is accomplished by retransmitting requests until a response is received.

One problem that arises from the model of application-level reliability is retransmission storms. If an application does not respond quickly enough (within 500 ms by default in the specification), the request will be retransmitted. This retransmission continues until a response is received. Each retransmit makes more work for the receiving server. Hence, retransmissions can cause more retransmissions, and thrashing can ensue.

A GC cycle that takes longer than 500 ms will cause retransmissions. One that takes several seconds will ensure many retransmissions. These, in conjunction with the new requests that arrive at the server during garbage collection, will make even more work for the server and it will fall further behind.

Even absent the overburdening problems just described, other constraints make garbage collection pauses unacceptable. There are carrier grade requirements that state acceptable latencies from the moment an initiating side begins a call to the moment it receives a response from the terminating side. These are typically sub-second times, so a multiple-second pause in a SIP server for GC is unacceptable.

5.2 Call Setups

"Call setup" [17] is a concept used throughout this paper. A call setup is simply the combination of an originating side stating that it wishes to initiate a session (a call in this case) and a terminating side responding that it is interested as well. In SIP, there is also the concept of an acknowledgement being sent back to the terminating side after it accepts.

After a call setup is complete, the server must maintain call setup state for 32 seconds in order to handle properly any application-level retransmissions that might occur. This specification is the reason the value of 32 seconds is used as an active duration throughout the paper.

5.3 Improvements

The original SIP server was able to achieve an execution efficiency of only about 65% on a four-way machine. By using the GC analyzer and making code changes appropriately, its developers boosted execution efficiency to about 90%.

Perhaps more importantly, the server now has much more consistent and predictable behavior, and is much faster as well.

Latencies have been reduced to the point that they are no longer an issue. Long gone are the days of the monolithic mark-compact collector taking more than 10 seconds to complete the collection of a 768-MB heap. Now maximum stop-the-world GC times are measured below 200 ms – perfect for a near-real-time application like a SIP server.

6. Modeling Application Behavior to Predict GC Pauses

Modeling Java applications makes it possible for developers to remove unpredictability attributed to the frequency and duration of pauses for GC.

The frequency and pause duration of GC are directly related to the following factors:

- Incoming call-setup rate
- Number and total size of objects created per call setup
- Average lifetime of these objects
- Sizes of the Java heaps

Developers can construct a theoretical model to show application behavior for various call-setup rates, numbers of objects created, object sizes, object lifetimes, and heap sizes. A theoretical model reflects the extremities of the application behavior for the best conditions and the worst. Once developers know these extremities, they can model a real-life scenario that helps predict the maximum call-setup rate that can be achieved with acceptable pauses, and that shows what can happen when call-setup rates exceed the maximum and application performance deteriorates.

6.1 Best-Case Scenario

Assumptions:

- Calculations are for a SIP application, but could be applied to any generic client-server application that has a request, a response, and an active time that request state is maintained
- For simplicity, only call setups are modeled (call tear-downs are not considered)
- The young collector is configured with the promoteall switch
- The old generation uses a concurrent inc-mark-sweep collector
- The variables that change from scenario to scenario are listed in a table preceding the results

Time call-setup state is maintained – active duration of call setup	0 ms
Time call-setup state is maintained	0 ms
Time between call setups	10 ms
Young semi-space size	5MB
Young GC threshold	100%
Old heap size	N/A
Old GC threshold	N/A
Total size of objects per call setup	50 KB
Lifetime of objects	0 ms

```
A young generation GC (GC[0], see 7.2 Snapshot of a GC Log) occurs when the 5-MB semi-space is 100% full.
Ideal objects space taken / call setup:

    = (semi-space size / total size of each call setup's objects)
    = 5 MB / 50 KB
    = 102.4 call setups / semi-space

    -> (10)
So a GC occurs every:

    frequency   = (call setups * call-setup rate)
        = 102.4 * 10
        = 1,024 ms

    -> (11)
So a GC occurs every:
```

```
    period      = (call setups * time between call setups)
        = 102.4 * 10
        = 1,024 ms

    -> (11)
Remember, in this scenario, we are not accumulating any garbage, because we assume that all objects are very
short-lived and they are all collected every young GC. Hence, nothing is ever promoted to the old heap. So,
that each young GC takes 50 ms.
So in an hour of execution the application spends this much time in young GC:

    = (seconds per hour * (GC pause in ms / 1000))
    = 3600 * (50 / 1000)
    = 180 seconds, or 3 minutes

    -> (12)
So in an hour of execution the application spends roughly this much time in young GC:

    = (seconds per hour * (GC pause in ms / 1000))
    = 3600 * (50 / 1000)
    = 180 seconds, or 3 minutes

    -> (12)
Because no objects are promoted to the older generation, the concurrent collector is not activated, and its
the application performance is assumed to be zero.
Total processing time available / hour:

    = (Minutes in a hour) - (time lost to young generation collection)
    = 60 - 3
    = 57 minutes.


Total call setups per hour @ 100 calls / second:

    = (call setups per second) * (seconds per minute) *
          (number of valid minutes in a hour)
    = 100 * 60 * 57
    = 342,000 call setups / hour on 1 CPU


Total call setups per hour @ 100 calls / second:

    = (call setups per second) * (seconds per minute) *
       (number of minutes available to the application in an hour)
    = 100 * 60 * 57
    = 342,000 call setups / hour on 1 CPU


Serial portion from (12) is 3 minutes lost to GC every hour.

    = 3 / 57
    = 0.0526


Scalability with a four-CPU machine:

    Speedup = 1 / (F + ((1 -F) / N)                  <- From Amdahl's law
    Speedup = 1 / (0.0526 + (1 - 0.0526) / 4)
    Speedup = 1 / 0.289 = 3.45

    Efficiency = Speedup / N
    Efficiency = (3.45 / 4) = 86.37%


So at 86.37% efficiency, we can process:

    (342,000 * 0.8637) * 4 = 1,181,542 call setups / hour
```

6.2 Worst-Case Scenario

Assumptions:

Time call-setup state is maintained – active duration	Infinite
Time call-setup state is maintained	Infinite
Time between call setups	10 ms
Young semi-space size	5 MB
Young GC threshold	100%
Old heap size	512 MB
Old GC threshold	32%
Old GC threshold	68%
Total size of objects per call setup	50 KB
Lifetime of objects	Infinite
Lifetime of objects	Infinite

```
Because all objects in this scenario are long-lived and the promoteall option has been specified, at every
young-generation GC all the objects are promoted. Because the young GC threshold is 100%, 5 MB of objects wi
So to fill up a 5 MB semi-space:

    From (#10), we should have received about 102.4 calls


A GC occurs every:

    frequency   = (cps * call-setup rate)
        = 102.4 * 10
        = 1,024 ms

    -> (21)
A GC occurs every:

    period      = (cps * time between call setups)
        = 102.4 * 10
        = 1,024 ms

    -> (21)
Each collection promotes 5 MB, 5,242,880 Bytes  -> (22)
So if the application is run with an old heap of 512 MB:

    The old heap should fill up in about:

          = (heap size) / (size of objects being promoted)
          = 512 MB / 5 MB
          = 536,870,912 / 5,242,880
          = 102 promotions

    -> (23)
So if promoting 5 MB of objects to the old generation takes the young GC collector about 200 ms (because you
more expensive when promotion takes place, we assume a higher value than before), then we have:

    total pause duration = (number of promotions * pause duration)
          = 102 * 200
          = 20,400 ms
          = 20.4 seconds

    -> (24)
A promotion takes place every 1,024 ms, so for 102 promotions:

    time for promotions = (number of promotions * (frequency + pause duration))
          = 102 * (1,024 + 200)
```

```
          = 124,848 ms
          = 2.08 minutes

    -> (25)
A promotion takes place every 1,024 ms, so for 102 promotions:

    time for promotions = (number of promotions * (periodicity + pause duration))
          = 102 * (1,024 + 200)
          = 124,848 ms
          = 2.08 minutes

    -> (25)
With one CPU the heap should fill up in about 2.08 minutes.
Serial percentage due to young-generation GC:

    = ((total pause duration * 100) / (60 * (total time for promotions))
    = 20.4 * 100 / (60 * 2.08)
    = 16.34%

    -> (26)
Note: The serial percentage due to the old generation collector is assumed to be zero, because the amount of
spent in stop-the-world GC for the concurrent collector is negligible compared to the amount of time spent i
stop-the-world young GC.
With four CPUs:

    Speedup = 1 / (.1634 + (1 - .1634) / 4) = 2.673

    -> (27)

    Execution Efficiency = (2.673 / 4) * 100 = 66.93%

    -> (28)
So with 4 CPUs, we should fill up the heap in about:

    = (#23) * (#26)
    = (2.08 * 0.6693) = 83.52 seconds

    -> (29)
```

6.3 A Real Scenario with Each Call Setup Active for 32 Seconds

(Calculated for Concurrent GC)

Assumptions:

Time call-setup state is maintained – active duration	32,000 ms
Time call-setup state is maintained	32,000 ms
Time between call setups	10 ms
Young semi-space size	5 MB
Young GC threshold	100%
Old heap size	512 MB
Old GC threshold	32%
Old GC threshold	68%
Total size of objects per call setup	50 KB
Lifetime of objects	50% = 0 ms 50% = 32,000 ms

```
Each young-generation GC promotes about 2.5 MB of live objects, because half the objects are short-lived and
To fill up the 5-MB semi-space:
```

```
    From (10), 102.4 call setups


A GC occurs every:

    = (call setups received * call-setup rate)
    = 102.4 * 10 = 1,024 ms

    -> (31)
A GC occurs every:

    = (call setups received * time between call setups)
    = 102.4 * 10
    = 1,024 ms

    -> (31)
A promotion promotes:

    = 2.5 MB of live data               <- assumption (30)
    = 2,621,440 Bytes

    -> (32)
So a promotion occurs every 1,024 ms, so in 32,000 ms, the number of promotions:

    = (active duration of call setup / frequency)
    = 32,000 / 1,024
    = 31 promotions

    -> (33)
So a promotion occurs every 1,024 ms, so in 32,000 ms, the number of promotions:

    = (active duration of call setup / periodicity)
    = 32,000 / 1,024
    = 31 promotions

    -> (33)
32,000 ms is the active duration of a call setup;

    the first call setup will be released at the end of the 32,000 ms
    so in 32,000 ms the number of call setups that can be received:

    = (active duration of a call setup / callsetup rate)
    = 32,000 / 10
    = 320 calls

So one active-duration segment is made up of 320 calls.     -> (34)
32,000 ms is the active duration of a call setup;

    the first call setup will be released at the end of the 32,000 ms
    so in 32,000 ms the number of call setups that can be received:

    = (active duration of a call setup / time between call setups)
    = 32,000 / 10
    = 3,200 calls

So one active-duration segment is made up of 3,200 calls.   -> (34)
At the end of 64,000 ms, all call setups from the first active duration will be dead.
At the end of 96,000 ms, call setups from two active durations will be dead.
At the end of 128,000 ms, call setups from three active durations will be dead.
At the end of 160,000 ms, call setups from four active durations will be dead.
In each young GC we promote:

    = 2,621,440 Bytes               <- from (32),


So total promotion in bytes every 32 seconds is:

    = (#32) * (#33)
    = 2,621,440 * 31
    = 81,264,640 or 79.36 MB
```

```
    -> (35)
The initial mark GC will occur when only 32% of the heap is free:

    = (32% * 512 MB)
    = 163.84 MB free
    = 512 MB - 163.84 MB
    = 348.16 MB used

    -> (36)
The initial mark phase will occur when the heap is 68% full:

    = (68% * 512 MB)
    = 348.16 MB used

    -> (36)
Time when an initial mark GC might occur is:

    = (initial mark size in MB / promotion size in MB) * frequency of promotion
    = (348.16 MB / 2.5 MB) * 1,024 ms
    = 142,606 ms

    -> (37)
Time when an initial mark phase might occur is:

    = (initial mark size in MB / promotion size in MB) * periodicity of promotion
    = (348.16 MB / 2.5 MB) * 1,024 ms
    = 142,606 ms

    -> (37)
After an initial mark GC, a remark GC takes place; assume that this happens at 28%:

    = ((100% - 28%) * 512 MB)
    = 368.64 MB used

    -> (38)
After an initial mark phase, a remark phase takes place; assume that this happens when the heap is 72% full:

    = ((100% - 28%) * 512 MB)
    = 368.64 MB used

    -> (38)
Time when a remark GC might occur is:

    = (remark size in MB / promotion size in MB) * frequency of promotion
    = (368.64 MB / 2.5 MB) * 1,024
    = 150,995 ms

    -> (39)
Time when a remark phase might occur is:

    = (remark size in MB / promotion size in MB) * periodicity of promotion
    = (368.64 MB / 2.5 MB) * 1,024
    = 150,995 ms

    -> (39)
Number of active durations present in the old heap at remark GC:

    = ((#38) / (#35))
    = 368.64 / 79.36
    = 4.6

    -> (40)
Number of active durations present in the old heap at remark phase:

    = ((#38) / (#35))
    = 368.64 / 79.36
    = 4.6

    -> (40)
```

```
Time to fill up an active-duration segment:

    = (#33) * (#31)
    = 31 * 1,024
    = 31,744 ms

    -> (41)
Time to fill up an active-duration segment:

    = (#33) * (#31)
    = 31 * 1,024
    = 31,744 ms

    -> (41)
Time to fill up 4.6 active durations:

    = 31,744 * 4.6 = 146,022 ms


The time when a resize GC can take place is soon after a remark GC:

    = (#39)
    = 150,995 ms

    -> (42)
The time when a resize phase can take place is soon after a remark phase:

    = (#39)
    = 150,995 ms

    -> (42)
The number of active-duration segments a resize GC can clean up:

    = ((#42) / active duration of a call setup)
        - (adjust factor for current active-duration segment)
    = (150,995 / 32,000) - 1
    = 3.72 Active-duration segments

    -> (43)
The number of active-duration segments that will be freed at the end of the resize phase:

    = ((#42) / active duration of a call setup)
        - (adjust factor for current active-duration segment)
    = (150,995 / 32,000) - 1
    = 3.72 Active-duration segments

    -> (43)
The resize GC should free 3.72 active durations:

    = (#43) * (#35)
    = 3.72 * 79.36 = 295.22 MB

    -> (44)
The resize phase should free 3.72 active durations:

    = (#43) * (#35)
    = 3.72 * 79.36 = 295.22 MB

    -> (44)
A young-generation GC should occur every 1,024 ms, and an old-generation initial mark phase should occur eve
```

7. Modeling Real-time Application Behavior Using "verbose:gc" Logs

The model above is theoretical and relies on a lot of assumptions; but developers can use the "verbose:gc" log from an actual application run to construct a real-world model. The model will allow one to visualize the runtime behavior of both the application and the garbage collector. The verbose:gc logs contain valuable information about garbage-collection times, the frequency of collections, application run times, number of objects created and destroyed, the rate of object creation, the total size of objects per call, and so on. This information can be analyzed on a time scale, and the behavior of the application can be depicted in graphs and tables that chart the different relationships among pause duration,

frequency of pauses, and object creation rate, and suggest how these can affect application performance and scalability. Analysis of this information can enable developers to tune an application's performance, optimizing GC frequency and collection times by specifying the best heap sizes for a given call-setup rate (see 14. Sizing the Young and Old Generations' Heaps).

7.1 Java "verbose:gc" Logs

Logs containing detailed GC information are generated when a Java application is run with the `-verbose:gc` flag turned on. For the 1.2.2_08 VM on Solaris, specifying this switch on the command line twice results in even more verbose logs – the level of output that the GC analyzer expects as input:

```
java -Xgenconfig:8m,8m,semispaces:512M,512M,incmarksweep
 -verbose:gc -verbose:gc program
```

7.2 Snapshot of a GC Log

The snapshot below is of a `verbose:gc` log entry generated by the JDK 1.2.2_08 VM when the `-verbose:gc` switch was specified twice on the command line. Highlighted phrases with superscript numbers are footnoted below.

```
java version "1.2.2"
Solaris VM (build Solaris_JDK_1.2.2_08, native threads, sunwjit)
semispaces
csp0 : data = fac00000 : limit = fb000000: reserve = fb000000
csp1 : data = fb000000 : limit = fb400000: reserve = fb400000

Starting GC at Mon Jun  4 11:42:03 2001; suspending threads.
Gen[0](semi-spaces):
                   size=8Mb
                    1       (50% overhead), free=0kb, maxAlloc=0kb.
space[0]:
                   size=4096kb  2                   , free=0kb, maxAlloc=0kb.
space[1]: size=4096kb
                   (100% overhead 3)                   , free=0kb,
maxAlloc=0kb.
Gen0(semi-spaces)-GC #11
                   tenure-thresh=31                      4 18ms                       5
                   0%->82%                       6                     free
Gen[0](semi-spaces): size=8Mb(50% overhead), free=3370kb,
maxAlloc=3370kb.
                    (from-space) space[0]: size=4096kb(100% overhead), free=0kb, maxAlloc=0kb. (
                    to-space)  space[1]: size=4096kb, free=3370kb, maxAlloc=3370kb. GC[
                    0]                        7 in 19ms                        8: (48Mb, 87% free) ->
                    (48Mb, 94% free)                       9 [application 179 ms                        10
                    requested 52 words]
                    Total GC time: 410 ms
                     11 ++ GC added 0 finalizers++ Pending finalizers = 0                       12
```

1. Total size for young-generation semi-spaces

2. Size of one semi-space

3. GC threshold

4. Tenure-thresh: "31" is the maximum number of times an object can be copied between the semi-spaces. In this snippet no objects have been promoted to the older generation but some have been copied to the to-space.

5. Copy time: time to copy live objects from from-space to to-space or time to promote objects to the older generation.

6. Percentage of to-space free

7. GC[0]: young-generation copy collector collection cycle

8. Pause time: time to stop application threads, tenure live objects, resume application threads

9. Total heap space free, including old-generation size

10. Time application was run before a tenure GC began
11. Cumulative GC time
12. Finalizer information (not used in this study)

8. Using the GC Analyzer Script to Analyze "verbose:gc" Logs

The GC analyzer is a script that analyzes the verbose:gc log on a time scale (wall clock time), and builds a mathematical model. The script provides:

- GC-related information
- Call-setup information
- Active-duration information
- Application-behavior information
- Various verifications

Here is snapshot of its output:

```
Application info:

        Application run time = 235,000 ms
        Memory = 272 MB
        Semispace = 16,384 KB

Young GC --------- (Tenured GC + Promoted GC) ---

Tenured GC info:

        Total number of tenure GCs = 0
        Average Object size tenured = 0 bytes
        Periodicity of tenure GC  = 0 ms
        Copy time = 0 ms
        Percent of app. Time = 0%

 Promoted GC info:

        Total number of promoted GCs = 187
        Average number of objects promoted = 66,761
        Average Objects size promoted = 4,436,045 bytes
        Periodicity of promoted GC  = 1,107.3 ms
        Promotion time = 142.3 ms
        Percent of app. Time = 11.89%

 Young GC info:

        Total number of young GCs = 187
        Average GC pause = 149.4 ms
        Copy/promotion time = 142.3 ms
        Overhead (suspend, restart threads) time  = 7.1 ms
        Periodicity of GCs = 1,107.3 ms
        Percent of app. Time = 11.89%

Old concurrent GC info:

        Total GC time = 1,710 ms
        Total number of GCs = 46
        Average Pause = 37.2 ms
        Periodicity of GC = 5,108.7 ms

Old mark-sweep GC info:

        Total GC time = 0 ms
        Total number of GCs = 0
        Average pause = 0 ms

Total old (concurrent + ms) GC info:
```

```
        Cost of concurrent GC = 68,000 ms
        Percent of app. Time = 33.11%

        Total GC time = 29,643 ms
        Average Pause = 127.2 ms

Call control info:

        Call setups per second (CPS) = 90
        Call rate, 1 call every = 11.11 ms
        Number of call setups / young GC = 99.7
        Total call-setup throughput = 18,636.03
        Total size of objects /
        call setup = 168,348 bytes
        Total size of short lived
        objects / call setup = 123,835 bytes
        Total size of long live
        objects / call setup = 44,513 bytes
        Total size of objects created
        per young gen GC = 16,777,216 bytes
        Average number of Objects
        promoted / call = 670

Execution efficiency of application:

        GC Serial portion of application = 12.61%
        Speedup = 2.90
        Execution efficiency = 0.7255
        CPU Utilization = 72.55%
```

The detailed output is shown in Appendix A.

The information generated by the analyzer can be used to tune application performance in the following ways:

- Reducing GC collection times
- Reducing the frequency of the young and old GCs
- Detecting memory leaks
- Detecting the rate of creation and destruction of objects, and using it to optimize the call setup rate
- Knowing the actual lifetimes of objects
- Sizing of the young and old generation heaps
- Modeling application and GC behavior
- Determining the scalability and execution efficiency of an application

9. Reducing Garbage Collection Times

Java applications have two types of collections, young-generation and old-generation.

9.1 Young-Generation Collection Times

A young-generation collection occurs when a semi-space is full. A copying collector is used to copy (or tenure) the live objects of one semi-space to the other semi-space. When that semi-space becomes full, the live objects could either be copied ("tenured") back to the original semi-space or promoted to the old generation, if they have already aged sufficiently. When developers inspect the GC logs they will find two types of young-generation GCs, called tenure GCs and promotion GCs.

9.1.1 Snippet from a Tenure GC

```
Starting GC at Mon Jun  4 11:42:04 2001; suspending threads.
Gen[0](semi-spaces):
                size=8Mb                    1     (50% overhead), free=0kb,
maxAlloc=0kb.
space[0]:
```

```
                  size=4096kb                      2    , free=0kb, maxAlloc=0kb.
space[1]: size=4096kb
                  (100% overhead  3)                   , free=0kb, maxAlloc=0kb.
                   Gen0(semi-spaces)-GC #11 tenure-thresh=31                     4
                  18ms                      5
                  0%->82%                      6 free
Gen[0](semi-spaces): size=8Mb(50% overhead),
free=3370kb, maxAlloc=3370kb.
space[0]: size=4096kb(100% overhead), free=0kb, maxAlloc=0kb.
space[1]: size=4096kb, free=3370kb, maxAlloc=3370kb.
                   GC[0]                     7                    in
                  19 ms                     8                   : (48Mb, 87% free) ->
                  (48Mb, 94% free)                     9
                   [application 179 ms                      10
                   requested 52 words]
Total GC time:
                  410 ms                      11
++ GC added 0 finalizers++ Pending finalizers = 0
```

1. Total size of young-generation semi-spaces
2. Size of first semi-space
3. GC threshold, 100%
4. Tenure-thresh: some of the intermediate objects have been copied to the to-space.
5. Copy time: time to copy live objects from from-space to to-space
6. Percentage of "to-space" free
7. GC[0], young generation, copy collector collection cycle
8. Pause time: time to stop application threads, tenure live objects, and resume application threads
9. Total heap space free, including old-generation size
10. Time application ran before a tenure GC began
11. Cumulative GC time

The above snippet shows tenuring (aging) of long-term objects. No promotions are done in a tenure GC. Live objects are copied back and forth between the two semi-spaces, allowing the short-term objects more time to die.

9.1.2 Snippet from a Promotion GC

```
Starting GC at Mon Jun  4 11:42:04 2001; suspending threads.
Gen[0](semi-spaces):
                    size=8Mb                       1   (50% overhead), free=0kb,
maxAlloc=0kb.

  space[0]:
                    size=4096kb                       2    , free=0kb, maxAlloc=0kb.
  space[1]: size=4096kb
                    (100% overhead       3)                     , free=0kb, maxAlloc=0kb.
                     Gen0(semi-spaces)-GC #11 tenure-thresh=31                        4
                    18ms                        5
                    0%->82%                        6 free

Gen[0](semi-spaces): size=8Mb(50% overhead),
free=3370kb, maxAlloc=3370kb.
  space[0]: size=4096kb(100% overhead), free=0kb, maxAlloc=0kb.
  space[1]: size=4096kb, free=3370kb, maxAlloc=3370kb.
                     GC[0]                       7                     in
                    19 ms                      8                    : (48Mb, 87% free) ->
                    (48Mb, 94% free)                          9
                     [application 179 ms                          10
                     requested 52 words]

Total GC time:
```

```
                    410 ms                          11
++ GC added 0 finalizers++ Pending finalizers = 0
```

1. Tenure-thresh: objects with age > 5; about 6% have been promoted.

2. Copy time + promotion: time to tenure (copy) long-term objects from from-space to to-space, plus time to promote (copy) objects to the older generation

3. Number of long-term objects promoted

4. Size of long-term objects promoted

The above snippet shows promotion of long-term objects. Live objects, which survive tenuring, are copied to the old-generation heap space. For a promotion GC, the line "promoted xxxx obj's" will report the number of objects, and the total size of the objects promoted.

The snippet also shows evidence of tenuring. Objects still alive when young GC occurs have been tenured, while older objects, about 6%, have been promoted.

The young-generation collection time is directly proportional to the size of the semi-spaces, the number of objects created, and the lifetimes of these objects. A collection, and hence a pause, occurs when a semi-space is full. Live objects are tenured, or promoted if the object has aged sufficiently. The cost of a tenure GC is the time to copy live objects to the second semi-space, while the cost of a promotion GC is the time to promote or copy live objects to the old generation. The cost of promoting is a little higher than that of tenuring, because the young collector has to interact with the old collector's heap.

Decreasing the semi-space size increases the frequency of young-generation collection and decreases the collection duration, as less live data has accumulated. Similarly, increasing the semi-space size decreases the frequency of young-generation collection and increases the duration of each collection, as more live data accumulates. Note, though, that less frequent collection also provides more time for the short-term data to die. It also partly offsets system overhead like thread scheduling, done at every collection cycle. The net result is a slightly longer collection but at a decreased frequency.

To tune the young generation collection time, developers must determine the right combination of frequency, collection time, and heap size. These three parameters are directly affected by the number of objects created per call setup and the lifetimes of these objects. So in addition to sizing the heap, the code may need to change, to reduce the number of objects created per call setup, and also to reduce the lifetimes of some objects.

The following output from the GC analyzer helps with this task:

- Average pause per young-generation GC
- Rate of promotion
- Average promotion size
- Number of tenure GCs
- Frequency of tenure GCs
- Number of promotion GCs
- Frequency of promotion GCs

See Appendix A for the detailed output.

9.1.3 Using This Information to Tune an Application

9.1.3.1 Using the promoteall Modifier

Three categories of object lifetime are important to the decision whether to use the `promoteall` modifier:

- Temporary objects – die before encountering even one young GC
- Intermediate objects – survive at least one young GC, but are collected before promotion
- Long-term objects – live long enough to get promoted

Temporary objects are never tenured or promoted because they die almost instantly. Long-term objects are always promoted, because they live through multiple young GCs. Only intermediate objects benefit from tenuring: After the first time they are tenured, they usually die and are collected in the next cycle, sparing the collector the cost of promoting them.

If the application has few intermediate objects, using the `promoteall` modifier decreases the amount of time that the application spends in young GC. This saving comes from not copying long-term objects from one semi-space to the other before promoting them to the old heap anyway.

Intermediate objects that are promoted take slightly more time to promote than to tenure. Also, the old-heap collector collects these objects, so it again uses slightly more time to collect them when they do die. However, the old-heap collection happens concurrently rather than in the young collector's stop-the-world fashion, so scalability should improve.

9.1.3.2 promoteall Modifier Usage

```
java -Xgenconfig:8m,8m,semispace,
 promoteall
:128m,128m,incmarksweep program
```

9.1.3.3 Snippet of a Promotion GC with the promoteall Modifier

```
Starting GC at Fri Jun  8 15:16:16 2001; suspending threads.
Gen[0](semi-spaces): size=12Mb(50% overhead), free=0kb, maxAlloc=0kb.
  space[0]: size=6144kb, free=0kb, maxAlloc=0kb.
  space[1]: size=6144kb(100% overhead), free=0kb, maxAlloc=0kb.

                   Gen0                      1                    (semi-spaces)-GC #8234
                  tenure-thresh=0                        2
                  481ms                     3                     0%->94% free,
                   promoted 50805 obj's                     4/2731kb                      5

Gen[0](semi-spaces): size=12Mb(50% overhead), free=5760kb, maxAlloc=5760kb.
  space[0]: size=6144kb(100% overhead), free=0kb, maxAlloc=0kb.
  space[1]: size=6144kb, free=5760kb, maxAlloc=5760kb.
GC[0] in 486 ms: (518Mb, 20% free) -> (518Mb, 20% free) [application 264 ms
requested 36 words]
Total GC time: 980190 ms
```

1. Gen0: young-generation collection cycle
2. Tenure-thresh=0: no objects are tenured
3. Promotion: time to promote (copy) objects to the older generation
4. Number of long-term objects promoted
5. Size of long-term objects promoted

The above snippet shows all live objects being promoted, so tenure-thresh=0.

9.1.3.4 Tracking Young GCs that Tenure Objects

A review of the numbers in "9.1.1 Snippet from a Tenure GC" through "9.1.3.3 Snippet of a Promotion GC with the promoteall Modifier" will reveal how the tenure percentage changes from young GC to young GC. Without the `promoteall` modifier, this percentage reduces until there are more live objects in the semi-space than are allowed. At that time, these objects are promoted to the old heap, and the threshold is reset to its starting value.

There are times when tenuring can help application performance. When intermediate objects are given enough time that they become temporary objects, they are collected before they are ever promoted to the old heap. It is for this reason that tenuring is used in the first place. This strategy works well for many classes of application. Determining whether an application will benefit from tenuring, or from promoting all live objects to the old heap, will help developers configure applications for optimal behavior.

9.1.3.5 Reducing the Size of Promoted or Tenured Objects

Depending on the application's behavior, merely increasing the size of the semi-spaces may help by allowing long-term objects to become intermediate objects, and intermediate objects to become temporary objects (see 9.1.3.4 Tracking Young GCs that Tenure Objects). Increasing the

semi-spaces may help, but it does increase the time that each young collection takes.

The next step is code inspection. There are a few ways to reduce the time it takes to tenure or promote objects.

The easiest is to find objects that are kept alive longer than necessary, and to stop referring to them as soon as they are no longer needed. Sometimes, this is as simple as setting the last reference to `null` as soon as the object is no longer needed.

Also look for objects that are unnecessary in the first place, and simply don't create them. If, as is typical, these are temporary objects, avoiding their creation will help indirectly, by reducing the frequency of young GCs. If they are long-term or intermediate objects, the benefit is more direct: they need not be tenured or promoted.

Sometimes it is not as simple as just not creating an object. Developers may be able to reduce object sizes by making non-static variables static, or by combining two objects into one, thus reducing tenure or promotion time.

9.1.3.6 Disadvantages of Pooling Objects

In general, object pooling is not usually a good idea. It is one of those concepts in programming that is too often used without really weighing the costs against the benefits. Object reuse can introduce errors into an application that are very hard to debug. Furthermore, retrieving an object from the pool and putting it back can be more expensive than creating the object in the first place, especially on a multi-processor machine, because such operations must be synchronized.

Pooling could violate principles of object-oriented design (OOD). It could also turn out to be expensive to maintain, in exchange for benefits that diminish over time, as garbage collectors become more and more efficient and collection costs keep decreasing. The cost of object creation will also decrease as newer technologies go into VMs.

Because pooled objects are in the old generation, there is an additional cost of re-use: the time required to clean the pooled objects so they are ready for re-use. Also, any temporary objects created to hold newer data are created in the younger generation with a reference to the older generation, adding to the cost of young GCs .

Additionally, the older collector must inspect pooled objects during every collection cycle, adding constant overhead to every collection.

9.1.3.6 Advantages of Pooling Objects

The main benefit of pooling is that once these objects are created and promoted to the old generation, they are not created again, not only saving creation time but reducing the frequency of young-generation GCs. There are three specific sets of factors that may encourage adoption of a pooling strategy.

The first is the obvious one: pooling of objects that take a long time to create or use up a lot of memory, such as threads and database connections.

Another is the use of static objects. If no state must be maintained on a per-object basis, this is a clear win. A good general rule is to make as many of the application's objects and member variables static as possible.

When it is not possible to make an object static, imposing a policy of one object per thread can work just fine. Such cases are good opportunities to take advantage of a static `ThreadLocal` variable. These are objects that enable each thread to know about its own instance of a particular class.

Note that in JDK 1.2 the `ThreadLocal` implementation has some synchronization and object creation associated with it. This flaw was resolved in v1.3 and beyond. Developers using Java 1.2 may find it beneficial to implement subclasses of `Thread` and `ThreadLocal` that work like the 1.3 versions of these classes.

9.1.3.7 Seeking Statelessness

An application can achieve its maximum performance if the objects are all or mostly short-lived, and die while still in the young generation. To achieve such short lifetimes, the application must be essentially stateless, or at least maintain state for only a brief period. Minimizing statefulness helps greatly because the young generation's copying collector is very efficient at collecting dead objects. It expends no effort, just skips over them and goes on to the next object. By contrast any object that must be tenured or promoted imposes the cost of copying to a new area of memory.

9.1.3.8 Watching for References that Span Heaps

Developers should avoid creating ephemeral or temporary objects after objects are promoted to the old generation. If the application creates temporary objects that have references from or to objects in the old generation , then the cost of scanning them is greater than the cost of scanning objects whose only references are within the young generation.

9.1.3.9 Flattening Objects

Keeping objects flat, avoiding complex object structures, spares the collector the task of traversing all the references in them. The fewer references there are, the faster the collector can work – but note that trade-offs between good OOD and high performance will arise.

9.1.3.10 Watching for Back References

Look for unnecessary back references to the root object, as these can turn a temporary object into a long-lived one – and can lead to a memory leak, as the reference may never go away.

9.2 Old-Generation Collection Times

While the young-generation collector is a stop-the-world collector, the old generation's concurrent inc-mark-sweep collector runs concurrently with the application. It does, however, stop the application briefly to take snapshots of the heap. These snapshots are taken in three stages :

- Initial mark phase
- Remark phase
- Resize phase

9.2.1 Initial Mark Phase

When the free memory in the old heap drops below a certain percentage, usually between 40% and 32%, the old-generation collector starts an "initial mark" phase. The initial mark phase takes a snapshot of the heap, and starts traversing the object graph to distinguish the referenced and unreferenced objects. Marking, in a nutshell, is the process of tagging each object that can still be reached , so that it can be preserved when unmarked objects are collected.

9.2.1.1 Snippet of an Initial Mark Phase

```
Starting GC at Fri Jun 8 15:12:23 2001; suspending threads.
GC[1] 1: initial mark 2] in 25 ms 3:
(518Mb, 32% free 4) ->
(518Mb, 32% free) [application 2 ms requested 0 words]
Total GC time: 877635 ms
```

1. GC[1]: old generation's concurrent inc-mark-sweep collection phase
2. Initial mark: old generation's initial mark snapshot
3. Pause time: time used to take the snapshot
4. Percentage of heap free when the snapshot was taken

9.2.2 Remark Phase

Once the concurrent mark phase is complete, the "remark" phase begins. The collector re-walks parts of the the object graph that may have changed since they were scanned intially.

9.2.2.1 Snippet of a Remark Phase

```
Starting GC at Fri Jun  8 15:12:33 2001; suspending threads.
              GC[1                1             :
             remark                2]              in
             68 ms                3:
              (518Mb, 27% free)                4              -> (518Mb, 27% free)
[application 190 ms  requested 0 words]
Total GC time: 882274 ms
```

1. GC[1]: old generation's concurrent inc-mark-sweep collection phase
2. Remark: old generation's remark snapshot
3. Pause time: time to take the snapshot
4. Percentage of heap free when the snapshot was taken

9.2.3 Resize Phase

The "resize" GC phase follows the remark phase. In this phase the collector checks to see whether there is enough space. If there is not, it resizes the heap.

9.2.3.1 Snippet of Resize Phase

```
Starting GC at Fri Jun  8 15:12:39 2001; suspending threads.
              GC[1                1: resize heap                2]
in
             1 ms                3             :
             (518Mb, 78% free)                4              ->
(518Mb, 78% free) [application 225 ms  requested 0 words]
Total GC time: 884199 ms
```

1. GC[1]: old generation's concurrent inc-mark-sweep collection phase
2. Old generation's resize phase
3. Pause time: time to resize or free old-generation memory
4. Percentage of heap that is free after the resize

Warning: If the object creation rate is too high, the concurrent collector will be unable to keep up with the collection. The free threshold may drop below 5%, and the old generation, instead of using the concurrent inc-mark-sweep collector, will employ the traditional mark-sweep stop-the-world collector to collect the older heap.

9.2.4 Snippet of Traditional Mark-Sweep GC

```
Starting GC at Fri Jun  8 15:16:54 2001; suspending threads.
Gen[0](semi-spaces): size=12Mb(50% overhead),
    free=2kb, maxAlloc=2kb.
  space[0]: size=6144kb, free=2kb, maxAlloc=2kb.
  space[1]: size=6144kb(100% overhead),
      free=0kb, maxAlloc=0kb.
Gen0(semi-spaces)-GC #8266 tenure-thresh=0
     748ms 0%->94% free, promoted 50616 obj's/2960kb
Gen[0](semi-spaces):
    size=12Mb(50% overhead), free=5760kb, maxAlloc=5760kb.
  space[0]: size=6144kb(100% overhead), free=0kb, maxAlloc=0kb.
  space[1]: size=6144kb, free=5760kb, maxAlloc=5760kb.
                   Gen[1]                      1

                  (inc-mark-sweep                       2)   : size=512Mb, free=16kb, maxAlloc=16kb.

Gen1(inc-mark-sweep)-GC #71
                  6123ms                        3
                   5%                        4                    ->
                  38%                        5                     free
Gen[1](inc-mark-sweep): size=512Mb, free=19kb, maxAlloc=19kb.
GC[1] in
                  6876                         6                     ms:
    (518Mb, 5% free) -> (518Mb, 38% free)
    [application 321 ms requested 1028 words]
Total GC time: 1010530 ms
```

1. Gen[1]: old generation's traditional inc-mark-sweep collection phase
2. Inc-mark-sweep GC
3. Sweep time: time to sweep or free the old generation's memory
4. Percentage of heap free when the mark-sweep GC began
5. Percentage of heap free after the sweep

6. Pause time of the mark-sweep GC, including stopping all other threads (including the young collector's thread), completing the mark-sweep phase, and resuming threads

9.2.5 Observations Regarding Old-Generation Collections

9.2.5.1 Undersized Old Heaps

After each resize phase, the size of the heap collected should be equal to or greater than the size of an active duration. Depending on the heap size, more than one active-duration segment could be in the old generation. Based on the time of the active duration, and the frequency of the old-generation GC, all of the objects in the active duration could be collectable.

The GC analyzer provides this ratio, (size of active durations / resize heap space). This value should be less than 1. A value greater than 1 indicates that, when an old generation GC takes place, there are active-duration segments still alive. With the ratio greater than 1, the call rate could be putting pressure on the older generation, forcing on it more frequent collections. If the ratio is too high, it can actually force the old generation to employ the traditional mark-sweep collector. Simply increasing the size of the old-generation heap can alleviate this pressure. Note, though, that too great an increase could lead to a smear effect (14.2.4 Locality of Reference Problems with an Oversized Heap).

The GC analyzer is able to determine the pressure on the older collector, as is described in "11. Detection of Memory Leaks."

9.2.5.2 Checks and Balances

At the end of an active duration, the amount of heap resized should be equal to the size of the objects promoted. If there are lingering references or if the timers are not very sensitive to the actual duration, then a memory leak may result. Memory leaks will fill up the old heap, degrading the old collector's performance and increasing the frequency of the old-generation GC. The GC analyzer reports such behavior.

For a final verification, at the end of the application run, the total size of the old generation objects collected should be compared to the total size of the objects promoted. These should be approximately equal. See "11. Detection of Memory Leaks" for more details.

10. Reducing the Frequency of Young and Old GCs

The frequency of the young GC is directly related to:

- The semi-space size
- The call-setup rate
- The rate of object creation
- The objects' lifetimes

10.1 The Size of the Semi-space

Increasing the semi-space reduces young-GC frequency but increases the time each collection takes, because the promotion size could be higher. On the other hand, a ratio of temporary to long-lived objects that is very high could decrease the collection time, as more intermediate objects could die before a tenure GC or a promotion GC occurs. Changing the semi-space size entails a trade-off between decreasing frequency and increasing collection times. See "9.1 Young-Generation Collection Times" and "9.2 Old-Generation Collection Times" for details on the effects of a change in the semi-space size on frequency and collection time.

10.2 The Call-Setup Rate and the Rate of Object Creation

Frequency of GC is directly proportional to the call-setup rate and to the size of the objects created per call setup. The GC analyzer reports, per call setup, the total size of objects created, and of the temporary and long-term data created, and the average object size. This information can be used to reduce unnecessary object creation and optimize the application code.

10.3 Object Lifetimes

The frequency of young GC is also dependent on the lifetimes of the objects created. Long lifetimes lead to unnecessary tenuring in the young generation. Using the `promoteall` modifier could alleviate this problem, by shifting to the old generation's collector the burden of collecting, sooner or later, all objects alive at the beginning of any young GC. For details on using the `promoteall` modifier, see "9.1.3.2 promoteall Modifier Usage".

The lifetimes of objects have a big impact on application performance, as they affect GC frequency, collection times, and the young and old heaps. If objects never die, then at some time there will be no more heap space available and an out-of-memory exception will arise.

Increased object lifetimes reduce old-generation memory available and lead to more frequent old-generation GCs. Object lifetimes also affect the young generation, because the promotion size increases as lifetimes increase, and in turn increases collection times. For maximum performance, references to objects should be nulled when they are no longer needed.

11. Detection of Memory Leaks

The analyzer determines the number of active call setups currently alive in the old heap. It does so by taking into account the active duration of call setups and the call-setup rate. If the application is well behaved, the entire active-duration segment should be freed at the end of the live time of the call setups. The analyzer verifies this by calculating the size of the heap that is freed and correlating this information with the size of the active-duration segments that should have been freed. Some of the ratios are shown below; for example:

```
CPS = 200// Call setups per second
Call-setup rate = 1 call every 5 ms
Active duration of each call setup = 32,000 ms
Number of call setups in active duration = 6,400
Number of promotions in active duration = 79.74
Long-lived objects (promoted objects)       -> (40)
/ active duration = 109,334,972 bytes       -> (41)
Number of active durations freed by old GC = 6.28       -> (42)
Average resized memory size = 687,152,824 bytes
Ratio of live to freed data = 0.83
Time when init GC might take place = 109,267 ms         -> (43)
Periodicity of old GC = 166,000 ms      -> (44)
```

From the ratios above, the number of active durations that should ideally be freed would be:

```
= (periodicity of old GC / active duration of each call setup)
= ((#44) / (#40))
= (166,000 / 32,000) = 5.185        -> (45)
```

From above, the number of active durations freed (#42) is 6.28, which indicates call setups are being freed, and old-heap free memory is in good shape. If the ratio (#42) / (#45) is very low, less than 1, then the old-heap sizing should be inspected, or object references are lingering even after the call setup that generated them is no longer active.

Note: A developer should size the heap to hold at least two or more active durations, so that when an old GC takes place, at least 1 or more active durations are freed. From the model above, (#42 = #43 / #41), 6.3 active durations were dead at the end of 160,000 ms. Comparing this to the theoretical or ideal number that could be dead at the time of old GC (#45) reveals that the application is freeing its references properly.

A good way to detect memory leaks is to compare the total size of the objects promoted to the total size of the objects reclaimed for the entire test. Any substantial difference signals a leak. Memory leaks lead to various problems:

- Increase in frequency of old GC
- Increase in collection times
- Execution of the less efficient mark-sweep collector
- An out-of-memory exception

Ratios from the GC analyzer for memory verification:

```
Total size of objects promoted:
    = 1,661,646 KB      -> (51)

Total size of objects fully GC collected in application run:
    = 1,307,750 KB      -> (52)
```

The difference should be one or more active-duration segments:

```
= (#51) - (#52)
= 1,661,646 - 1,307,750
= 353,896 KB        -> (53)

= (#53) / ((#42) / (#41)
= (353,896 * 1,024) / (687,152,824 / 5.22)
= 3.31 active durations
```

12. Finding the Optimal Call-Setup Rate by Using the Rates of Creation and Destruction

Using the call-setup rate as input, the analyzer provides various call-setup rate calculations, which can be used to determine the following:

- Average number of objects created per call setup
- Average size of each object
- Total size of temporary objects
- Total size of long-lived objects

If the application is creating too many objects, either temporary or long-lived, developers should devise a strategy to maintain an optimum ratio of temporary to long-lived objects. This tuning will help reduce the young GC collection time and the frequency of collection.

Based on the size and number of objects promoted to the old generation, developers can adopt a strategy of sizing the heap to accommodate the desired maximum call-setup rate. A case for pooling could also be made, if the average number of the long-lived objects is large, and it takes many computational cycles to create the objects for each call setup.

```
Call setups per second (CPS) = 200
Call-setup rate, 1 call every = 5 ms
Call setup before young gen GC = 80.26
Size of objects / call setup = 78,390 bytes
Size of short-lived objects / call = 61,307 bytes
Size of long-lived objects / call setup = 17,084 bytes
Ratio of young GC (short/long) objects / call setup = 3.59
Average number of objects promoted / call setup = 367
```

13. Learning the Actual Object Lifetimes

It is very important to know the lifetime of an object, whether it is temporary or long-lived. Because objects are associated with call setups, knowing the real lifetime will make it possible to size both the young– and the old-generation heaps properly. If this could be exactly calculated, then the young-generation semi-space could be sized to destroy as many short-lived objects as possible, and promote only the necessarily long-lived objects. The `promoteall` modifier could be used if the ratio of temporary objects to long-lived objects is low.

Knowing the real lifetime also helps developers size the old-generation heap optimally, to accommodate the desired maximum call-setup rate. It also helps detect memory leaks by identifying objects that linger beyond their necessary lifetime. The GC analyzer can currently detect active-duration segments but not individual object lifetimes. The JVMPI (Java Virtual Machine Profiler Interface) [22] can be used to report the lifetime of individual objects, and this information can then be integrated with the output of the GC analyzer to verify heap usage accurately.

14. Sizing the Young and Old Generations' Heaps

Heap sizing is one of the most important aspects of tuning an application, because so many things are affected by the sizes of the young– and old-generation heaps:

- Young-GC frequency
- Young-GC collection times
- The number of short-term and long-term objects
- The number of call setups received before promotion
- Promotion size
- Number of active-duration segments
- Old-GC frequency

- Old-GC collection times
- Old-heap fragmentation
- Old-heap locality problems

14.1 Sizing the Young Generation

The young generation's copying collector does not have fragmentation problems, but could have locality problems, depending on the size of the generation. The young heap must be sized to maximize the collection of short-term objects, which would reduce the number of long-term objects that are promoted or tenured. Frequency of the collection cycle is also determined by heap size, so the young heap must be sized for optimal collection frequency as well.

Basically, finding the optimal size for the young generation is pretty easy. The rule of thumb is to make it about as large as possible, given acceptable collection times. There is a certain amount of fixed overhead with each collection, so their frequency should be minimized.

14.2 Sizing the Old Generation

The concurrent inc-mark-sweep collector manages the old-generation heap so it needs to be carefully sized, taking into account the call-setup rate and active duration of call setups. An undersized heap will lead to fragmentation, increased collection cycles, and possibly a stop-the-world traditional mark-sweep collection. An oversized heap will lead to increased collection times and smear problems (14.2.4 Locality of Reference Problems with an Oversized Heap). If the heap does not fit into physical memory, it will magnify these problems.

The GC analyzer helps developers find the optimal heap size by providing the following information:

- Tenure– and promotion-GC information
- Old-generation concurrent inc-mark-sweep collection times
- Traditional-GC information
- Percentage of time spent in young– and old-generation GC

Reducing the percentage of time spent in young and old GC will increase application efficiency.

14.2.1 An Undersized Heap and a High Call-Setup Rate

As the call-setup rate increases, the rate of promotion increases. The old heap fills faster, and the number of active-duration segments rises. Frequency of old-generation GCs increases because the older generation fills up faster. If the pressure on the old collector is heavy enough, it can force the old collector to revert to the traditional mark-sweep collector. If the old collector is still unable to keep up, the system can begin to thrash, and finally, throw an out-of-memory exception.

Increasing the old-generation heap size can alleviate this pressure on the old collector. The heap should be enlarged to the point that a GC collects from one to three active-duration segments. Use the following information from the GC analyzer to determine whether the heap size is configured properly:

- Initial mark threshold
- Remark threshold
- Number of bytes collected by the resize phase
- Number of active-duration segments collected in the collection
- Number of active-duration segments promoted

14.2.2 An Undersized Heap Causes Fragmentation

An undersized heap can lead to fragmentation because the old-generation collector is a mark-sweep collector, and hence never compacts the heap. When the collector frees objects, it combines adjacent free spaces into a single larger space, so that in can be optimally allocated for future objects. Over time, an undersized heap may begin to fragment, making it difficult to find space to allocate an object. If space cannot be found, a collection takes place prematurely, and the concurrent collector cannot be used, because that object must be allocated immediately. The traditional mark-sweep collector runs, causing the application to pause during the collection. Again, simply increasing the heap size, without making it too large, could prevent these conditions from arising.

If the heap does fragment, and the call-setup rate slows, then the fragmentation problem will resolve itself. As more and more objects die and are collected, space in the heap becomes available again. Take the extreme example, where there are no call setups for a period of time greater than an active duration. Now all call-setup objects will be collected at the next GC, and the heap will no longer be fragmented.

14.2.3 An Oversized Heap Increases Collection Times

When using the concurrent inc-mark-sweep collector, the cost of each collection is directly proportional to the size of the heap. So an oversized heap is more costly to the application. The pause times associated with the collections will still not be that bad, but the time the collector spends collecting will increase. This does not pause the application, but does take valuable CPU time away from the application.

To aid its interaction with the young collector, the old heap is managed using a card table, in which each card represents a subregion of the heap. Increasing the heap size increases the number of cards and the total size of the data structures associated with managing the free lists in the old collector. An increased heap size and a high call rate will add to the pressure on the old collector, forcing it to search more data structures, and thus increasing collection times.

14.2.4 Locality of Reference Problems with an Oversized Heap

Another problem with an oversized heap is "locality of reference." This is related to the operating system, which manages memory. If the heap is larger than physical memory, parts of the heap will reside in virtual memory. Because the concurrent inc-mark-sweep collector looks at the heap as one contiguous chunk of memory, the objects allocated and deallocated could reside anywhere in the heap. That some objects will be in virtual memory leads to paging, translation-lookahead-buffer (TLB) misses, and cache-memory misses. Some of these problems are solved by reducing the heap size so that it fits entirely in physical memory, but that still does not eliminate the TLB misses or cache-memory misses, because object references in a large heap may still be very far away from each other.

This problem with fragmentation and locality of reference is called a "spread," or "smear," problem. Optimizing heap sizes will help avoid this problem.

14.3 Sizing the Heap for Peak and Burst Configuration

Developers should test the application under expected sustained peak and burst configuration loads, to discover whether collection times and frequency are acceptable at maximum loads. The heap size should be set to enable maximum throughput under these most demanding conditions. Far better to discover the application's limitations *before* it encounters peak and burst loads during actual operation.

15. Determining the Scalability and Execution Efficiency of a Java Application

As was discussed in (4. How GC Pauses Affect Scalability and Execution Efficiency), the serial parts of an application greatly reduce its scalability on a multi-processor machine. One of the principal serial components of a Java application is its garbage collection. The concurrent inc-mark-sweep collection greatly reduces the serial nature of old-generation collection, but young-generation collection remains serial.

The GC analyzer calculates the effects of young- and old-generation GCs. It uses this information to determine the execution efficiency of the application. This, in turn, can be used to reduce the serial percentage and increase scalability. Output from the GC analyzer also helps developers evaluate other solutions, such as running multiple JVMs or using the concurrent collector.

15.1 GC Analyzer Snippet Showing the Cost of Concurrent GC and Execution Efficiency

```
Total old (concurrent + stop-the-world) GC info:

        Cost of concurrent GC = 68,000 ms
        Percent of application time = 33.11%

Execution efficiency of the application:

        GC Serial portion of application = 12.61%
        Speedup = 2.90
        Execution efficiency = 0.7255
        CPU Utilization = 72.55%
```

16. Other Ways to Improve Performance

Other means to improve application performance include using the Solaris RT (real-time) scheduling class or using the alternate thread library (/usr/lib/lwp), which is the default thread library on Solaris 9.

Experimenting with the JVM options that are available, and re-testing with various combinations until an optimal configuration is found, is one way to change application behavior and get more performance without making any code changes.

17. On the Horizon

JDK 1.4 does not include the advanced concurrent inc-mark-sweep collector. This collector and perhaps more advanced garbage-collection features should become available from JDK 1.4.1 onwards. The performance-tuning techniques discussed in this paper are applicable across JDK 1.3 and JDK 1.4. The behavior of the concurrent collector in JDK 1.4.1 should be similar to that seen in JDK 1.2.2_08. The verbose:gc log format will be different, so the model needs to be constructed using the new information.

18. Conclusion

Insight into how an application behaves is critical to optimizing its performance. A Java application's performance can be tuned and improved by a variety of means related to garbage collection. GC times can be reduced by employing the concurrent collector, and by using properly sized young- and old-generation heaps.

Garbage collection times can severely limit the scalability of a Java application. The concurrent inc-mark-sweep collector can be used improve scalability – but the constraints of the young generation's stop-the-world copying collector remain. In the future, parallel young-generation collectors may be available to ameliorate this problem. For now, running multiple JVMs can help take advantage of all the CPU power available on the machine.

Using the GC analyzer will enable developers to size the young– and old-generation heaps optimally. It will also point out what types of optimization might most fully enhance an application's performance. By analyzing, optimizing, and re-analyzing, developers can see which variations improve application performance. This is a continuous, iterative process that should eventually yield optimal performance from each application.

Using advanced garbage collectors can greatly enhance application behavior. It can even improve scalability on multiprocessor machines.

19. Acknowledgments

We would like to thank the Java VM, GC team, (Y.S. Ramakrishna, Ross Knippel), and Sun Labs (Steve Heller) for helping us with the internals of the concurrent GC collector. This work and the analysis would not have been possible without significant help from them. We would also like to thank our colleagues in Market Development Engineering (IPE) at Sun. From dynamicsoft, we would like to thank Jon Schlegel, Allan Andersen and the Java User Agent (UA) engineering team for contributing to this effort.

Finally, we would like to thank Brian Christeson for polishing this paper with some sharp proofing and edits.

20. References

- David Detlefs, William D. Clinger, Matthias Jacob, and Ross Knippel, "Concurrent Remembered Set Refinement in Generational Garbage Collection", USENIX JVM'02, http://www.usenix.org/events/javavm02/tech.html
- Basic Java 2
- Hot Spot Whitepaper
- Monica Pawlan
- Jacob Seligmann and Steffen Grarup, "Incremental Mature Garbage Collection Using the Train Algorithm"
- Richard Jones, "The Garbage Collection page," http://www.cs.ukc.ac.uk/people/staff/rej/gc.html
- Thomas W. Christopher and George K. Thiruvathukal, "High-Performance Java Platform Computing"
- Hot Spot
- Steve Wilson, "Java Platform Performance Strategies and Tactics"
- James Gosling, Bill Joy, and Guy Steele, "The Java Language Specification, "
- Tony Eyers, and Henning Schulzrinne, "Predicting Internet Telephony Call Setup Delay", http://www.cs.columbia.edu/~hgs/papers/Eyer0004_Predicting.pdf (PDF)
- M. Handley, H. Schulzrinne, E. Schooler, and J. Rosenberg, "SIP: session initiation protocol," Request for Comments 2543, Internet Engineering Task Force, Mar. 1999. http://www.ietf.org/rfc/rfc2543.txt
- R. Fielding, J. Gettys, J. Mogul, H. Frystyk, L. Masinter, P. Leach, and T. Berners-Lee, "Hypertext Transfer Protocol -- HTTP/1.1," Request for Comments 2616, Internet Engineering Task Force, June 1999. http://www.ietf.org/rfc/rfc2616.txt
- Java Virtual Machine Profiler Interface (JVMPI)

Appendix A

A1. GC Analyzer Usage

Usage:

```
gc_analyze.pl [-d] <gclogfile>
<CPS> <active_call_duration> <CPUs>
<application_run_time_in_ms>
```

Parameter Name	Meaning
–d	Print out a detailed analysis and a summary
gclogfile	verbosegc log file to process
CPS	Call setups / second (default = 50)
active_call_duration	Duration call setup is active (default = 32,000 ms)
CPUs	Number of CPUs in the system (default = 1)
applcation_run_Time_in_ms	Number of ms application was run (default is calculated from GC time stamps)

A2. GC Analyzer Output with –d Option

Running the GC analyzer with the –d option generates detailed output, which includes a summary. The detailed output provides in-depth insight into application behavior.

```
solaris% gc_analyze.pl -d
    logs/data/re_gc_fi_200_4p_768_RT.log
200 32000 4

Processing logs/data/re_gc_fi_200_4p_768_RT.log ...

Call rate = 200 cps ...
Active call setup duration = 32,000 ms
Number of CPUs = 4


---- GC Analyzer Summary:
logs/data/re_gc_fi_200_4p_768_RT.log   ----


Application info:

        Application run time = 498,000 ms
        Memory = 774 MB
        Semispace = 6,144 KB

Young GC --------- (Tenured GC + Promoted GC) ---

 Tenured GC info:

        Total number of tenure GCs = 0
        Average object size tenured = 0 bytes
        Periodicity of tenure GC = 0 ms
        Copy time = 0 ms
        Percent of app. time = 0%

 Promoted GC info:

        Total number of promoted GCs = 1,243
        Average number of Objects promoted = 29,529
        Average objects size promoted =
            1,377,229 bytes
        Periodicity of promoted GC = 325.46 ms
        Promotion time = 74.14 ms
        Percent of app. time = 18.77%

 Young GC info:

        Total number of young GCs = 1,243
```

```
        Average GC pause = 75.18 ms
        Copy/promotion time = 74.14 ms
        Overhead (suspend, restart threads)
        time = 1.04 ms
        Periodicity of GCs = 325.46 ms
        Percent of app. time = 18.77%

Old concurrent GC info:

        Total GC time = 286.00 ms
        Total number of GCs = 9
        Average pause = 31.78 ms
        Periodicity of GC = 55,333.33 ms

Old traditional mark-sweep GC info:

        Total GC time = 0 ms
        Total number of GCs = 0
        Average pause = 0.00 ms

Total old (concurrent + ms) GC info:

        Cost of concurrent GC = 36,000.00 ms
        Percent of app. time = 8.91%

Total (young and old) GC info:

        Total GC time = 93,736.00 ms
        Average pause = 74.87 ms

Call control info:

        Call setups per second (CPS) = 200
        Call rate, 1 call every = 5 ms
        Number# call setups / young GC = 65
        Total call throughput = 80,910.00
        Total size of objects /
            call setup = 96,654 bytes
        Total size of short lived objects /
        call setup = 75,496 bytes
        Total size of long live objects /
        call setup = 21,158 bytes
        Total size of objects created per
        young gen GC = 6,291,456 bytes
        Average number# of Objects promoted / call = 453


Execution efficiency of application:

        GC Serial portion of application = 18.82%
        Speedup = 2.56
        Execution efficiency = 0.64
        CPU Utilization = 63.91%


 --- GC Analyzer End Summary ----------------


                   #--- Detailed and possibly confusing calculations; dig into  this for more info about wh


---- GC Log stats ...


Totals GC0: GCs=#1341, #young_tenure_GCs=0,
#young_promoted_GCs=1243
  Tenure avgs: thresh=0, time=0, free=0
  Promoted avgs: thresh=0, time=74.14, free=100,
  objects=29529, size=1344.95
  Promoted totals: size_total=1671774
```

```
Totals GC1: GCs=9, #initmark_GCs=3,
    #remark_GCs=3 #resize_GCs=3
  Init-Mark avgs: time=17, totalmem=774, %=54,
      app_time=4.33
  Remark avgs: time=78, totalmem=774, %=29.66,
      app_time=208
  Resize avgs: time=0.33, totalmem=774, %=84.66,
      app_time=143.33

Totals ms GC1: GCs=0
  ms avgs: time=0, %=0, app_time=0


 ---- Young generation calcs ...

Average young gen dead objects size /
GC = 4,914,226.25 bytes
Average young gen live objects size /
GC cycle = 1,377,229.74 bytes
Ratio of short lived / long lived for young GC = 3.56

Average young gen object size promoted = 1,377,229.74 bytes
Average number# of Objects promoted = 29,529.49
Total object promoted times  = 93,450 ms
Average object promoted times = 74.14 ms
Total object promoted GCs = 1243
Periodicity of object promoted GCs = 325.46 ms

Total tenure times = 0 ms
Total tenure GCs = 0
Average tenure GC time = 0 ms
Periodicity of tenure GCs = 0 ms

Total number# of young GCs = 1243
Total time of young GC  = 93,450 ms
Average young GC pause = 75.18 ms
Periodicity of young GCs = 325.46 ms


--- Old generation calcs ...

Total concurrent old gen times = 286 ms
Total number# of old gen GCs = 9
Total number# of old gen pauses with 0 ms = 3
Total number# of old gen GCs = 9
Total old gen GCs = 3 cycles -> 1 full GC = 3
Average old gen pauses = 31.77 ms
Actual average old gen pauses = 31.77 ms
Periodicity of old gen GC = 55,333.33 ms
Actual Periodicity of old gen GC = 55,333.33


--- Traditional MS calcs ...

Total number# mark sweep old GCs = 0
Total mark sweep old gen time = 0 ms
Average mark sweep pauses = 0 ms
Average free threshold = 0%
Total mark sweep old gen application time = 0 ms
Average mark sweep apps time = 0 ms


 ---- GC as a whole ...

Total GC time = 93,736
Average GC pause = 74.86
Actual average GC pause = 74.86
```

```
--- Memory or Heap calcs ...

Total memory = 774 MB
Resize phase thresh = 84.66%
Remark phase thresh = 29.66%
Initial Mark phase GC thresh (init mark)= 54%

Live objects per old GC = 118.68 MB
Dead objects per old GC = 425.70 MB
Ratio of (short/long) lived objects per old GC = 3.58


 --- Memory leak verification ...

 Total size of objects promoted = 1,671,774 KB
 Total size of objects full GC collected
 throughout app. run = 1,307,750.40 KB


--- Active duration calcs ...

Active duration of each call = 32,000 ms
Number# number of calls in active duration = 6,400
Number# of promotions in active duration = 98
Long-lived objects (promoted objects) /
active duration = 135,411,421.16 bytes
Short-lived objects (tenured or not promoted) /
active duration = 483,174,548 bytes
Total objects created / active duration = 618,585,969.23 bytes
Percent% long-lived in active duration = 21.89%
Percent% short-lived in active duration = 78.10%
Number# of active durations freed by old GC = 5.07
Ratio of live to freed data = 0.83
Average resized memory size = 687,152,824.32
Time when init GC might take place = 88,225.34 ms
Time when remark phase might take place = 134,895.28 ms
Periodicity of initial old GC = 166,000 ms
Periodicity of old GC = 162,385.78 ms
Periodicity of resize phase = 162,385.78 ms


 --- Application run times calcs ...

Total application run times during young GC = 403,519 ms
Total application run times during old GC = 1,067 ms
Total application run time = 404,586 ms
Calculated or specified app run time = 498,000 ms
Ratio of young (gc_time / gc_app_time) = 0.23
Ratio of young (gc_time / app_run_time) = 0.18
Ratio of old (gc_time / gc_app_time) = 0.26
Ratio of old (gc_time / app_run_time) = 0.00
Ratio of total (gc_time / gc_app_time) = 0.23
Ratio of total (gc_time / app_run_time) = 0.18
```

A2. GC Analyzer Download

About the Authors:

Nagendra Nagarajayya, has been working with Sun for the last 8+ years. Currently, he works as a Staff Engineer at Market Development Engineering (MDE). At MDE, he works with Independent Software Vendors (ISVs) in the tele-communications (telco) & Retail industry, on issues related to architecture, performance tuning, sizing and scaling, benchmarking, porting, etc. He is interested, and specializes in multi-threaded issues, concurrency and parallelism, HA, distributed computing, networking, and performance tuning, in Java and Solaris.

Nagendra has a MS from San Jose State University, published many papers, and has been awarded several patents. He is also a member of the Dean's List, and Tau Beta Pi.

Steve Mayer is a five-year veteran at dynamicsoft, the leading provider of mobile services infrastructure software. As dynamicsoft's Director of Technology, Steve led the development team to bring to market the first commercial SIP User Agent. Steve has pushed the envelope of server side Java computing. By using optimization techniques that he has pioneered, dynamicsoft's products are not only the industry standard for performance but run in the most advanced telecommunications networks in the world. Currently, he has been involved in scalability and performance engineering, with a special emphasis on garbage collection.

Steve graduated with a B.S. in Computer Science from the Rochester Institute of Technology.